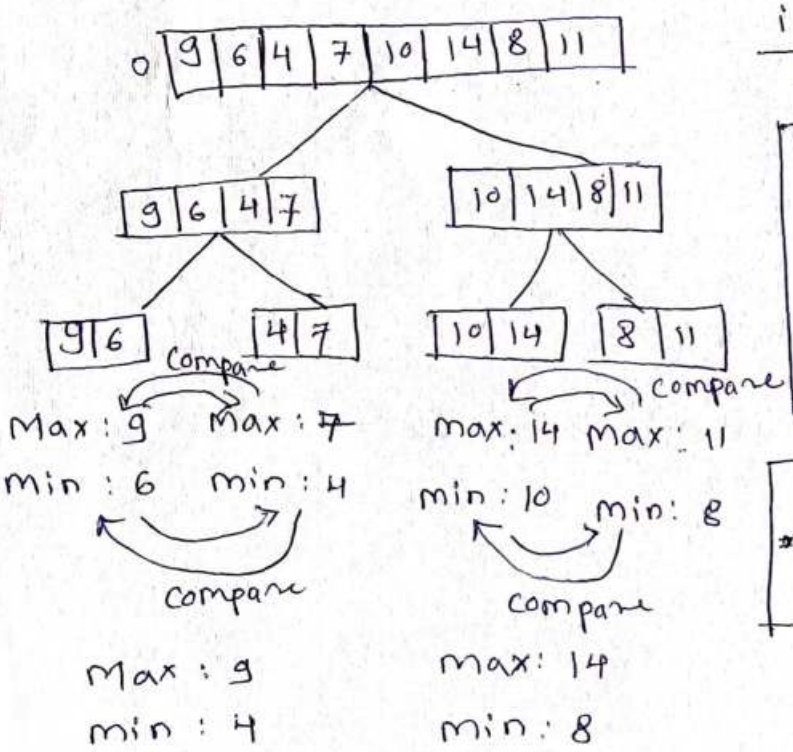
Min, Max without Divide & Conquer (DAC)

Linear Method

```
for i=2 to n do
if numbers [i] >= max then
max := numbers[i]
else
if number [i] <= min then
min := numbers [i]
return (max, min)
```

Min, Max with Divide & Conquer



$\frac{i+j}{2} = mid = \frac{1+8}{2} = \frac{9}{2} = 4.5$

$$T(n) = 2T(n/2) + 2$$

(2 comparision of max → max, min → min)

← Recurrence Relation of min-max using divide & conquer.

Back substitution.

$$O(n) = \frac{3n}{2} - 2$$

← To find out min-max (No. of comparisions)

Worst case

(linear) Approach $2n-2$

$\frac{3n}{2} - 2$ (DAC Approach)

Best Case

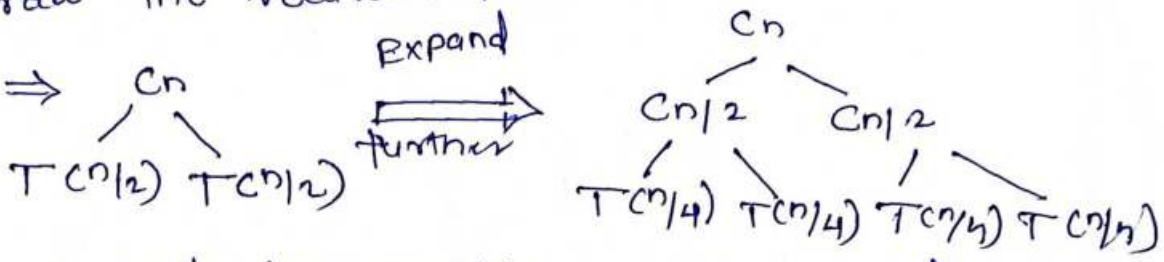
$n-1$

$\frac{3n}{2} - 2$

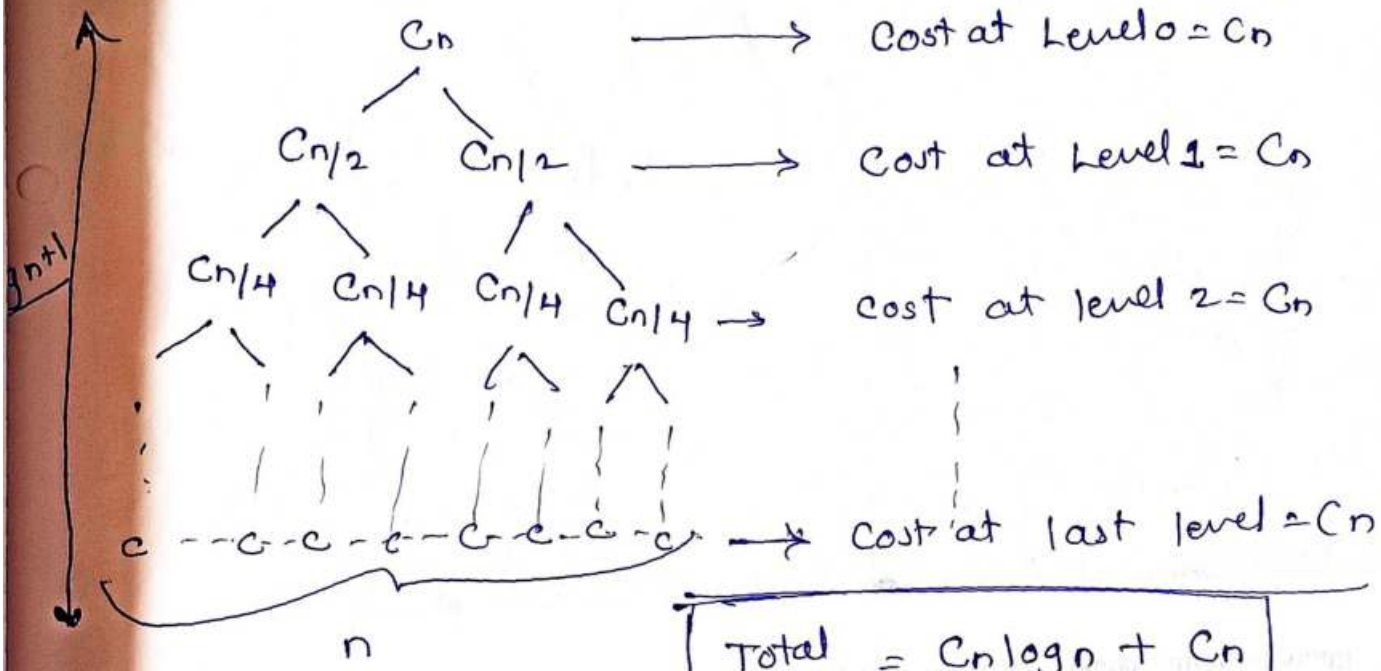
Merge Sort

Recurrence relation $T(n) = \begin{cases} c, & \text{if } n=1 \\ 2T(n/2)+Cn, & \text{if } n>1 \end{cases}$

Lets draw the recursive tree



so on .... here is the complete recursion tree diagram



$$\boxed{\text{Total cost} = Cn\log n + Cn}$$

Based on the above recursion tree, the cost at each level is $O(n)$, and there are $O(\log n)$ levels.
So the time complexity of the merge sort algo. is the sum of the

Term breakdown for the recurrence Relation:

$2T(\frac{n}{2})$: Represents how long it would take to sort the two sides of the array recursively. The array's division into two halves results in a factor of 2.

$O(n)$: Indicates how long it will take to combine the two sorted parts of the array since each element in the two halves must be compared and merged, merging requires linear time.

The Merge Sort algorithm's essence is encapsulated by the recurrence relation

$$T(n) = 2T(n/2) + O(n).$$

In Divide & Conquer Approach:

Step 1: Find the mid of the array

Step 2: Find the maximum & minimum of the left subarray recursively.

Step 3: Find the maximum and minimum of the right subarray recursively.

Step 4: Compare the result of step3 and step 4

Step 5: Return the minimum & maximum

Lets see the algorithm for the Divide and conquer Approach,

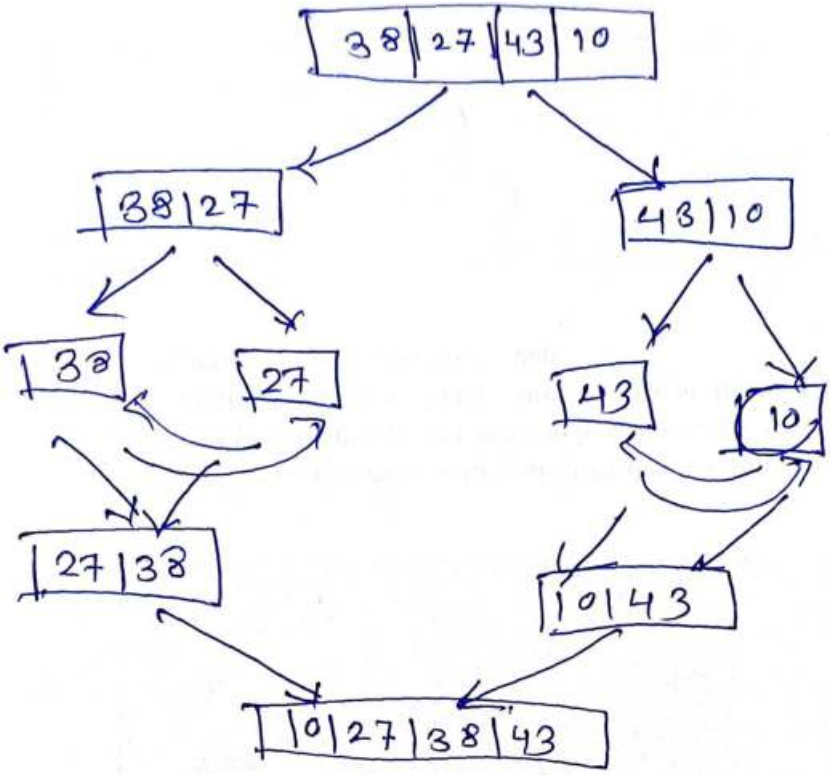
Suppose A is the array of integers and n is the number of elements in the array A.
i = 0 (Index of First element of array)
j = n-1 (index of last element of array)

```
MinMaxDAC (A, i, j) {
if (i==j)
    return (A[i], A[j]);
if (j-i) == 1)
    if (A[i] < A[j])
        return (A[i], A[j])
    else
        return (A[j], A[i])
else {
    int mid = (i+j)/2;
    LMin, LMax = MinMAXDAC (A, i, mid);
    RMin, RMax = MinMaxDAC (A, mid+1, j);
    if (LMax > RMax)
        max = LMax;
    else
        max = RMax;
    if (LMin < RMin)
        min = LMin;
    else
        min = RMin;
    return (min, max);
```

Time complexity for the DAC algo.
$T(n)=1$, if $n=1$ or $n=2$
$T(n) = 2T(n/2) + C$, if $n>2$
After solving the above recurrence relation.

$T(n) = \frac{3n}{2} - 2 \approx O(n)$

Merge Sort



Recurrence Relation of Merge Sort

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 2T(n/2) + \Theta(n) & \text{if } n > 1 \end{cases}$$

$T(n) \rightarrow$ Total time taken by the algorithm to sort an array of size n.

$2T(n/2) \rightarrow$ Time taken by the algo. to recursively sort the two halves of the array. Since each half has $n/2$ elements, we have two recursive calls with input size as $(n/2)$

$\Theta(n)$ represents the time taken to merge the two sorted halves.

Best, Avg & Worst case $\rightarrow O(n \log n)$

Binary search Algorithm :-

Linear search worst case = $O(\log n)$

target = 1

12	2	4	10	5	1

n

Binary search — $O(\log n)$

This approach is applied when array | vectors | data structures are sorted (whether ascending | descending)

sorted
asc des

arr = [-1, 0, 3, 4, 5, 9, 12]

less than mid more than mid target = 12

-1	0	3	4	5	9	12

st = 0 mid end = n-1

$mid = \frac{st + end}{2}$

tar > arr[mid]
2nd half

tar < arr [mid]
1st half

tar = arr [mid]
arr mid

In binary search, searching is the process of finding some particular element in the list. If the element is present in the list, then process is called successful & if the element is not present in the list, then the process is called unsuccessful.

FOR EDUCATIONAL USE

Sundaram

Binary search is the search technique that works efficiently on sorted lists. Hence, to search an element into some list using the binary search technique, we must ensure that the list is sorted. This approach follows divide and conquer approach in which the list is divided into two halves and the item is compared with the middle element of the list. If the match is found, then, the location of the middle elemenent is returned. Otherwise, we search into either of the halves depending upon the result produced through the match.

Algorithm of Binary search

Step 1

A → sorted Array
n → size of array
x → value to be searched

~~Set start st~~

Set st = lowerBound
Set end = upperBound
pos = -1

step1

~~While x not found~~
~~if upperBound < lowerBound~~
~~EXIT : x does not exist~~

Step3: set midPoint = lowerBound + upperBound / 2

Step 4: if a [midpoint] = val
set pos = midpoint
print pos

Step4: if a [midpoint] = val
set pos = midpoint
print pos

Go to step 6
else if a [midpoint] > value
set end = midpoint - 1
else
set beg = midpoint + 1
[end of if]
[end of loop]

Step 5: if pos = -1
print "value is not present in the array"

[end of if]

Step6 EXIT